Student Name: Juhi Surin
Register Number: 192525148
Course: CSA0919 – Programming in Java
Assignment: Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System
Task Number: Task 3
Task Title: Design the Control Logic
Marks: 4
Bloom's Level: K4 – Analyse
Java Version: Java 27

TASK 3 – Design the Control Logic (4 Marks)

---

1. Objective

This task designs the decision-making and looping logic for the two core operations of the Campus Smart-Parking System: allocating a suitable free parking slot and computing the parking fee. The allocation logic uses nested for loops, an if condition that tests for null, and the return statement. The fee logic uses if–else and switch statements to select the applicable rule. The task also explains the roles of break, continue and return, describes the behaviour when the facility is full, writes pseudocode for both operations before the Java code, completes the supplied ParkingFacility class, and analyses an alternative design that keeps free-slot counts and stacks.

2. Slot Allocation Logic

The facility stores its slots in a jagged array, slots[floor][slot]. An element holds the vehicle number of the parked vehicle, or null if the slot is free (as stated in the supplied program). The method allocate(String vehicleNo) works as follows.

- **Step 1 – Receive the vehicle number.** The method is called with the registration number of the entering vehicle.
- **Step 2 – Search the floors in order.** The outer loop runs floor from 0 to slots.length − 1. Here slots.length is the number of floors, so the floors are examined from the lowest to the highest.
- **Step 3 – Search the slots of the current floor.** The inner loop runs slot from 0 to slots[floor].length − 1. It uses the length of the current floor, so floors with different numbers of slots are handled correctly.
- **Step 4 – Test for a free slot.** The condition slots[floor][slot] == null identifies a free slot. A slot that holds a vehicle number is occupied and is skipped by the loop moving on to the next slot.
- **Step 5 – Store the vehicle number.** At the first null element, the statement slots[floor][slot] = vehicleNo marks the slot as occupied.
- **Step 6 – Return the position.** The method immediately executes return new int[]{floor, slot}, which gives the caller the floor index and the slot index.
- **Step 7 – Handle a full facility.** If both loops finish without finding a null element, every slot has been examined and none is free, so the method returns null.

Why return is used immediately after allocation. Once the first free slot has been found and filled, no further searching is needed, and continuing would be wrong because a second slot could be filled with the same vehicle. A single return statement ends both loops and the method at once and delivers the result. Because the floors and slots are searched in increasing order, the slot found is always the first free slot: the lowest floor, then the lowest slot number on that floor (the allocation policy assumed in Task 1).

Why null is returned for a full facility. The return type int[] must hold a floor and a slot, and every pair of valid values refers to a real slot. A full facility has no valid pair, so null is returned as the explicit value meaning "no slot", as the supplied comment specifies ("or null if it is full").

Precondition. allocate() assumes that vehicleNo is a valid, non-null registration number, which is checked before the call when the input is read (Task 2). Storing null would leave the slot looking free. The check that the same vehicle is not already parked (Task 1, rule BR2) is a separate rule and is not part of this method.

Code 1: Nested-loop allocation logic

```
public int[] allocate(String vehicleNo) {
    for (int floor = 0; floor < slots.length; floor++) {
        for (int slot = 0; slot < slots[floor].length; slot++) {
            if (slots[floor][slot] == null) {       // free slot found
                slots[floor][slot] = vehicleNo;       // occupy it
                return new int[]{floor, slot};        // stop at once
            }
        }
    }
    return null;                          // facility is full
}
```

3. Parking Fee Calculation Logic

What the assignment states and what is assumed. The Task 3 description gives the fee slabs only as examples ("first hour, each additional hour, daily maximum") and does not prescribe any hourly rate or any daily maximum. No numerical rate or maximum is therefore treated here as a requirement.

Table 1: Requirements stated in the scenario and assumptions in this report

Stated in the assignment	Assumed or illustrative in this report
Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge; visitors pay the full fee. EVs may use charging slots, which are charged separately. Fee slabs are named only as examples: first hour, each additional hour, daily maximum. No hourly rate or daily maximum is given for this task.	Every started hour is charged in full, and a stay of zero minutes costs nothing (Tasks 1 and 2). Slab amounts and a daily maximum are configuration values supplied by the university. They are applied only if they are defined. When a daily maximum is defined, it is applied to each block of 24 billable hours. A valid pass waives the parking fee only, not the charging fee (Task 1). Rates used in the Java illustration are placeholders to show the logic and are not requirements.

Selecting the fee rule with conditional statements. The decisions are taken in a fixed order, and a decision that settles the result ends the calculation early.

- **Pass check first.** If the user is a student or staff member and the pass is valid, the parking fee is zero and the duration does not need to be examined. An if statement tests the user category and the pass validity together.
- **Duration to billable hours.** Otherwise the duration in minutes is converted to billable hours with the ceiling division (durationMinutes + 59) / 60, so that every started hour is charged in full. Zero minutes gives zero hours and therefore a fee of zero.
- **Vehicle type selects the rate.** The vehicle type has a fixed set of values, so a switch statement selects the corresponding hourly rate (or the first-hour and additional-hour rates when slabs are used).
- **Slabs.** An if–else decides whether the stay is zero hours (fee zero) or one or more hours. With slabs, the fee is the first-hour rate plus (hours − 1) × the additional-hour rate. If both rates are equal, this reduces to the flat rule “hours × hourly rate”.
- **Daily maximum.** Only if a daily maximum is defined, the fee of each 24-hour block (and of the remaining hours) is limited to that maximum using Math.min. If none is defined, this step is skipped.
- **EV charging.** A separate if adds the charging fee for an electric vehicle after the parking fee has been decided.

if–else compared with switch. An if–else is used where a condition or a range is tested (pass validity, whether the hours are zero, whether a daily maximum exists). A switch is used where one variable is compared with a fixed set of constants (the vehicle type or the user category), which is clearer than a long chain of if–else statements.

Table 2: Parking fee by user category and pass validity

User category	Pass	Parking fee
Student	Valid monthly pass	Zero
Student	No pass or expired pass	Full fee for the billable hours

User category	Pass	Parking fee
Staff	Valid monthly pass	Zero
Staff	No pass or expired pass	Full fee for the billable hours
Visitor	Not applicable	Full fee for the billable hours

Parking charge compared with EV charging charge. The parking charge pays for occupying a slot. It depends on time (billable hours) and on the vehicle type, and a valid monthly pass can waive it. The EV charging charge pays for electricity supplied. It depends on the units consumed (kWh), applies only to electric vehicles, and is calculated and billed separately, so that the total is the parking fee plus the charging fee. A pass holder pays no parking fee but still pays for any charging used (assumption from Task 1).

Code 2: Illustrative conditional fee calculation (not part of ParkingFacility.java)

```
// Illustrative only. The rates are placeholders, not assignment values.
enum VehicleType { TWO_WHEELER, CAR, ELECTRIC_CAR }

static double ratePerHour(VehicleType type) {   // switch on a fixed set
    return switch (type) {
        case TWO_WHEELER -> 10.0;
        case CAR, ELECTRIC_CAR -> 30.0;
    };
}

static double parkingFee(int durationMinutes, VehicleType type,
        boolean validPassHolder) {
    if (durationMinutes < 0) {
        throw new IllegalArgumentException("negative duration");
    }
    if (validPassHolder) {          // eligible student or staff member
        return 0.0;
    }
    int hours = (durationMinutes + 59) / 60;   // every started hour
    return hours * ratePerHour(type);          // 0 minutes gives 0.0
}

static double chargingFee(VehicleType type, int kWh) {
    if (type != VehicleType.ELECTRIC_CAR || kWh <= 0) {
        return 0.0;                 // charged separately, EVs only
    }
    return kWh * 12.0;              // placeholder rate per kWh
}
```

Code 3: Illustrative slabs with an optional daily maximum (parameters, no fixed values)

```
static double slabFee(int hours, double firstHour, double additionalHour) {
    if (hours == 0) {
        return 0.0;
    }
    return firstHour + (hours - 1) * additionalHour;
}

static double cappedFee(int hours, double firstHour,
        double additionalHour, double dailyMaximum) {
    int fullDays = hours / 24;
    int remainingHours = hours % 24;
    return fullDays * Math.min(slabFee(24, firstHour, additionalHour), dailyMaximum)
        + Math.min(slabFee(remainingHours, firstHour, additionalHour), dailyMaximum);
}
```

4. Use of break, continue and return

break. The break statement leaves the nearest enclosing loop or switch at once when further processing is unnecessary. For example, when only the first free slot of a single floor is needed, the scan of that floor can stop as soon as it is found. In a classic switch statement, break also prevents execution from falling through into the next case.

```
int firstFree = -1;
for (int s = 0; s < slots[floor].length; s++) {
    if (slots[floor][s] == null) {
        firstFree = s;
        break;                  // stop scanning this floor
    }
}

switch (userCategory) {        // classic switch: break ends each case
    case "STUDENT": case "STAFF": category = "pass-eligible"; break;
    case "VISITOR": category = "pays full fee"; break;
}
```

continue. The continue statement skips the rest of the current iteration and goes straight to the next iteration of the loop. For example, when counting the free slots of a floor, an occupied slot has nothing to contribute and can be skipped.

```
int freeCount = 0;
for (int s = 0; s < slots[floor].length; s++) {
    if (slots[floor][s] != null) {
        continue;               // occupied: go to the next slot
    }
    freeCount++;                // reached only for a free slot
}
```

return. The return statement ends the method immediately and gives its result to the caller, even from inside nested loops. For example, a helper that reports whether a floor has a free slot can return as soon as one is found.

```
static boolean hasFreeSlot(String[] floorSlots) {
    for (int s = 0; s < floorSlots.length; s++) {
        if (floorSlots[s] == null) {
            return true;              // result known: leave the method
        }
    }
    return false;                 // every slot was occupied
}
```

Why return suits the first free slot. break would leave only the inner loop, so the outer loop would carry on to the next floor and a labelled break or a flag variable would be needed to stop it, followed by code to build the result. The return statement stops both loops, ends the method and delivers {floor, slot} in one step. It also guarantees that exactly one slot is filled and that the slot found is the first free one.

Note on the supplied allocate(). The completed allocate() uses return only. It does not use break or continue, and none is needed. The examples above illustrate the three statements in the parking context.

5. Handling a Full Parking Facility

When all parking slots are occupied, every element of slots[floor][slot] holds a vehicle number. The loops in allocate() then examine every slot of every floor without finding a null element, no slot is changed, and the method reaches its final statement, return null. A vehicle that cannot be allocated a slot therefore receives no floor and no slot, and no ticket should be issued for it (Task 1, rule BR4). The facility remains unchanged and consistent.

The caller must check the returned value before using it. If the result is null and the caller reads result[0] or result[1], the program throws a NullPointerException. The correct pattern is to test the value first:

```
int[] position = facility.allocate(vehicleNo);
if (position == null) {
    System.out.println("Facility full: no slot for " + vehicleNo);
} else {
    int floor = position[0];
    int slot = position[1];
    System.out.println("Allocated floor " + floor + ", slot " + slot);
}
```

6. Pseudocode for Both Core Operations

The pseudocode is written before the Java code. It uses the ALGORITHM … BEGIN … END structure, INPUT and OUTPUT definitions, capitalised keywords, indentation to show nesting, ← for assignment, // for comments, FOR, IF, ELSE and END IF, and RETURN statements.

Algorithm 1 – Allocate Parking Slot

```
ALGORITHM AllocateParkingSlot
INPUT:  vehicleNo – registration number of the entering vehicle
    slots – slots[floor][slot] holds a vehicle number, or NULL if free
OUTPUT: the pair (floor, slot) of the allocated slot, or NULL if full
BEGIN
    FOR floor ← 0 TO numberOfFloors − 1 DO
        FOR slot ← 0 TO numberOfSlotsOn(floor) − 1 DO
            IF slots[floor][slot] = NULL THEN
                slots[floor][slot] ← vehicleNo
                RETURN (floor, slot)
            END IF
        END FOR
    END FOR
    RETURN NULL      // every slot was examined and none is free
END
```

Algorithm 2 – Calculate Parking Fee

```
ALGORITHM CalculateParkingFee
INPUT:  durationMinutes – whole minutes parked, 0 or more
    vehicleType – TWO_WHEELER, CAR or ELECTRIC_CAR
    userCategory – STUDENT, STAFF or VISITOR
    passValid – TRUE if a valid monthly pass is held, else FALSE
    chargedKWh – units charged by an EV (0 if none), 0 or more
    CONFIGURATION (set by the university, not given by the assignment):
        firstHourRate, additionalHourRate, chargingRatePerKWh,
        dailyMaximum (or NOT DEFINED)
OUTPUT: parkingFee, chargingFee and totalFee
BEGIN
    IF durationMinutes < 0 OR chargedKWh < 0 THEN
        RETURN INVALID INPUT
    END IF
    // Part 1: parking fee
    IF (userCategory = STUDENT OR userCategory = STAFF) AND passValid = TRUE
    THEN
        parkingFee ← 0
    ELSE
        billableHours ← (durationMinutes + 59) DIV 60    // every started hour
        IF dailyMaximum IS NOT DEFINED THEN
            parkingFee ← SlabFee(billableHours)
        ELSE
            fullDays ← billableHours DIV 24
            remainingHours ← billableHours MOD 24
            parkingFee ← fullDays × MIN(SlabFee(24), dailyMaximum)
                + MIN(SlabFee(remainingHours), dailyMaximum)
        END IF
    END IF
    // Part 2: EV charging, calculated separately
    IF vehicleType = ELECTRIC_CAR THEN
        chargingFee ← chargedKWh × chargingRatePerKWh
    ELSE
        chargingFee ← 0
    END IF
    totalFee ← parkingFee + chargingFee
    RETURN (parkingFee, chargingFee, totalFee)
END
```

Algorithm 2a – Helper: SlabFee

```
ALGORITHM SlabFee
INPUT:  hours – number of billable hours, 0 or more
OUTPUT: fee for those hours using the rates of vehicleType
BEGIN
    IF hours = 0 THEN
        RETURN 0
    END IF
    RETURN firstHourRate + (hours − 1) × additionalHourRate
END
```

The rates firstHourRate and additionalHourRate are selected from the vehicle type (a switch in the Java code). If both are equal to one hourly rate, SlabFee returns hours × hourly rate, which is the flat rule of Tasks 1 and 2. The parking fee and the EV charging fee are calculated in separate parts and are only added in totalFee. A pass holder therefore has parkingFee = 0 but still has a chargingFee for any units charged.

Table 3: Behaviour of Algorithm 2 for typical inputs (rates are symbolic)

Input	billableHours	parkingFee	chargingFee
0 minutes, visitor, car	0	0	0
61 minutes, visitor, car	2	SlabFee(2)	0
61 minutes, student with valid pass, car	not needed	0	0
61 minutes, staff with valid pass, EV, 10 kWh	not needed	0	10 × chargingRatePerKWh
61 minutes, student with expired pass	2	SlabFee(2)	0

7. Completed Java Code

Only the two TODO sections of the supplied class were completed. The class name, the field, the constructor, the method signatures, the comments and the main() method are as supplied.

Code 4: ParkingFacility.java

```
public class ParkingFacility {
    // slots[floor][slot] holds a vehicle number, or null if the slot is free
    private final String[][] slots;

    public ParkingFacility(int[] slotsPerFloor) {
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            slots[f] = new String[slotsPerFloor[f]];
        }
    }

    /** Returns {floor, slot} of the first free slot, or null if it is full. */
    public int[] allocate(String vehicleNo) {
        // TODO 1: Search floor by floor and slot by slot; place vehicleNo in the first
        //   free slot and return {floor, slot}
        for (int floor = 0; floor < slots.length; floor++) {
            for (int slot = 0; slot < slots[floor].length; slot++) {
                if (slots[floor][slot] == null) {
                    slots[floor][slot] = vehicleNo;
                    return new int[]{floor, slot};
                }
            }
        }

        return null;
    }

    /** Frees a slot. Returns false if the position is invalid or already free. */
    public boolean release(int floor, int slot) {
        // TODO 2: Validate floor and slot, free the slot and return true; otherwise
        //   return false
        if (floor < 0 || floor >= slots.length) {
            return false;
        }
        if (slot < 0 || slot >= slots[floor].length) {
            return false;
        }
        if (slots[floor][slot] == null) {
            return false;
        }
        slots[floor][slot] = null;
        return true;
    }

    public static void main(String[] args) {
        // floor 0 has 2 slots, floor 1 has 1 slot
        ParkingFacility p = new ParkingFacility(new int[]{2, 1});
        System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " "
```

```
                + p.release(5, 0));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

TODO 1 – allocate(). The outer loop visits the floors in order and the inner loop visits the slots of the current floor using slots[floor].length. The first element equal to null is replaced by vehicleNo, and return new int[]{floor, slot} ends the method at once. The final return null is reached only if no free slot exists.

TODO 2 – release(). The method validates the position in three steps, each ending in return false: the floor must satisfy $0 \leq$ floor $<$ slots.length; the slot must satisfy $0 \leq$ slot $<$ slots[floor].length; and the slot must not already be empty (slots[floor][slot] == null). Only if all three checks pass is the slot set to null and true returned.

Validation order. The floor is checked before the slot because the slot is checked against slots[floor].length, and reading slots[floor] with an invalid floor would throw ArrayIndexOutOfBoundsException. Because each failed check returns immediately, slots[floor] is evaluated only after the floor is known to be valid. In the same way, slots[floor][slot] is read for the emptiness check only after both indexes have been validated.

8. Expected Output and Explanation

The expected output stated in the assignment is:

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

The facility is created with new int[]{2, 1}: floor 0 has two slots and floor 1 has one slot, so there are three slots in total, all initially null.

Table 4: Explanation of each output line

Line	Output	Why it occurs
1	[0, 0]	allocate("TN01AB1111"): floor 0, slot 0 is null, so the vehicle number is stored there and {0, 0} is returned immediately.
2	[0, 1]	allocate("TN02CD2222"): floor 0, slot 0 is now occupied and is skipped; floor 0, slot 1 is null, so it is filled and {0, 1} is returned.
3	[1, 0]	allocate("TN03EF3333"): both slots of floor 0 are occupied, so the inner loop ends and the outer loop moves to floor 1, where slot 0 is null; {1, 0} is returned.
4	null	allocate("TN04GH4444"): all three slots are occupied, so both loops finish without finding a null element and the method returns null. Arrays.toString prints a null array as null.

Line	Output	Why it occurs
5	true false false	One println statement evaluates three calls from left to right. release(0, 1) succeeds because slot 1 of floor 0 holds TN02CD2222, which is set to null (true). The second release(0, 1) finds the slot already null (false). release(5, 0) is rejected by the first check because floor 5 is not less than slots.length, which is 2, so no slots[5] access happens (false).
6	[0, 1]	allocate("TN04GH4444") scans again: floor 0, slot 0 is still occupied, but floor 0, slot 1 was freed by the first release, so the vehicle takes the first free slot, {0, 1}.

9. Execution and Validation Instructions

The program must be compiled and run on your own computer with Java 27 installed. The steps for VS Code are:

- **Save the file.** Create a folder named Task03 (inside your repository) and save the program in it as ParkingFacility.java. The file name must match the class name exactly, including capital letters.
- **Open the folder.** In VS Code choose File > Open Folder and select Task03, so that ParkingFacility.java is visible in the Explorer panel.
- **Open the terminal in this folder.** Choose Terminal > New Terminal (or press Ctrl + `). The terminal path must end with Task03. If it does not, change to it with cd followed by the folder path.
- **Check the Java version.** Run java -version and javac -version and confirm that both report version 27.
- **Compile.** Run javac ParkingFacility.java. No message means success, and the file ParkingFacility.class is created in the folder.
- **Execute.** Run java ParkingFacility (without .class).
- **Verify.** Compare the six lines printed with the expected output in Section 8, line by line. They must be identical, including the spaces in true false false.
- **Take the screenshot.** Keep the terminal window showing the commands javac ParkingFacility.java and java ParkingFacility and the six output lines, and capture it (on Windows press Windows + Shift + S and select the terminal area). Save the image as output.png in the Task03 folder.

```
javac ParkingFacility.java
java ParkingFacility
```

The result may be recorded as verified only after these steps have been carried out and the actual output has matched the expected output.

10. Alternative Concept – Free-Slot Counts and Stacks

The question. Instead of scanning every floor and slot until a free slot is found, suppose the system keeps a count of the free slots on each floor and, for each floor, a stack of its free slot

numbers. This design is for analysis and comparison only; it is not implemented in ParkingFacility.java.

Data structures. The slots array is kept to record which vehicle occupies each slot. In addition, freeCount[f] holds the number of free slots on floor f, and freeStacks.get(f) holds the free slot numbers of floor f. At the start, freeCount[f] equals the capacity of floor f, and each stack holds all slot numbers of its floor (pushed in descending order so that the lowest number is on top).

Allocation. The system scans the array of floor counts from floor 0 upwards and selects the first floor whose count is greater than zero. It pops a slot number from that floor's stack, stores the vehicle number in slots[floor][slot] to mark the slot as occupied, decreases the count of that floor by one, and returns {floor, slot}. If every count is zero, the loop ends and the method returns null, because all floors have no free slot.

Release. The system validates the floor and slot exactly as the original release() does. It rejects an empty slot, which prevents a duplicate release: without this check the same slot number would be pushed twice and could later be allocated to two vehicles. For a valid occupied slot it sets slots[floor][slot] to null, pushes the slot number onto that floor's stack, increments that floor's count and returns true.

Code 5: Illustrative allocation and release with counts and stacks

```
private final String[][] slots;
private final int[] freeCount;                  // free slots per floor
private final List<ArrayDeque<Integer>> freeStacks = new ArrayList<>();
// constructor: freeCount[f] = slotsPerFloor[f]; push slot numbers
// slotsPerFloor[f] - 1 down to 0 onto the stack of floor f

public int[] allocate(String vehicleNo) {
    for (int f = 0; f < freeCount.length; f++) {       // O(F) scan of counts
        if (freeCount[f] > 0) {
            int s = freeStacks.get(f).pop();               // O(1)
            slots[f][s] = vehicleNo;
            freeCount[f]--;
            return new int[]{f, s};
        }
    }
    return null;                                // all counts are zero
}

public boolean release(int floor, int slot) {
    if (floor < 0 || floor >= slots.length) return false;
    if (slot < 0 || slot >= slots[floor].length) return false;
    if (slots[floor][slot] == null) return false;     // no duplicate release
    slots[floor][slot] = null;
    freeStacks.get(floor).push(slot);               // O(1)
    freeCount[floor]++;
    return true;
}
```

Effect on the allocation policy. A stack returns the most recently released slot first. The scanning method always returns the lowest free slot. For example, on one floor of three slots, after releasing slot 0 and then slot 1, scanning allocates slot 0 but the stack returns slot 1. The

two designs give the same results for the supplied main() method, but they can differ in general. Keeping the lowest-slot rule would need a structure ordered by slot number, such as a priority queue, whose operations cost O(log S).

11. Complexity Analysis

Let F be the number of floors and S the maximum number of slots on any floor. The total number of slots is at most F × S. The occupancy array slots itself needs O(F × S) space in both designs and is not counted as auxiliary space.

Table 5: Scanning compared with free-count and stack

Criterion	Original scanning approach	Free-count and stack approach
Data structures used	The jagged array slots[floor][slot] only.	The slots array, an int array of F counts, and F stacks of free slot numbers.
Allocation time	O(F × S) in the worst case, when the facility is full or only the last slot is free; O(1) in the best case.	O(F): a linear scan of the F floor counts (O(F)) plus a stack pop (O(1)). It is not O(1) because the floors are still searched linearly.
Release time	O(1), assuming direct indexing and validation.	O(1): validation, a stack push and a count update, assuming direct indexing. No resizing occurs after construction because a stack never holds more than S numbers.
Auxiliary space	O(1), excluding the slots array.	O(F × S) for the stacks plus O(F) for the counts, which is O(F × S) in total.
Advantages	Simple; no extra state to keep consistent; always allocates the lowest free slot; matches the supplied method.	Allocation no longer examines individual slots; a full facility is detected from the counts alone.
Limitations	Allocation time grows with the number of slots examined.	Extra memory; counts, stacks and the slots array must always agree; most recently freed slot is reused first unless a priority queue is used; the floor scan is still linear.

Assumptions. Worst-case time is stated for a floor-major scan, array access and a null test cost O(1), the stack operations pop and push cost O(1), and constant factors are ignored. Initialising the original array and building the stacks both take O(F × S) once, at construction. This task gives an analytical comparison only; no timing or memory measurements are reported.

Which design is suitable. The original scanning design is the more suitable for this assignment. The facilities have at most 5 floors × 100 slots (500 slots), so a scan is fast enough, and the design needs no extra state that could become inconsistent. It also follows the supplied method structure exactly, always allocates the lowest free slot, and produces the expected output. The count-and-stack design would become worthwhile for a much larger facility with frequent allocations, where avoiding the slot-by-slot scan matters more than the added memory and complexity.

12. Result

The completed ParkingFacility class is designed to demonstrate allocation of the first available slot, correct release of occupied slots, rejection of invalid floor and slot indexes, rejection of the release of an already empty slot, correct handling of a full facility by returning null, and successful reallocation of a slot after it is released. The pseudocode for slot allocation and fee calculation, the explanation of break, continue and return, and the comparison with the free-count and stack design were completed as required.

Execution record (to be completed by the student after running the program):

Check	Done
Program compiled with javac ParkingFacility.java on Java 27	[ ]
Program executed with java ParkingFacility	[ ]
Actual output matched the expected output line by line	[ ]
Screenshot saved as output.png in the Task03 folder	[ ]